Machine Learning [CSE374s]

Credit Card Default (Taiwan)

Member Name	ID
Abdelrahman Hany Ashraf (role 1)	2300130
Ahmed Ayman Mohamed (role 2)	2300145
Yassin Ayman Elsayed (role 3)	2301087

Introduction

What is Machine Learning?

Machine learning is a field of science that helps computers learn from data. Instead of giving the computer direct rules, we provide it with historical data and the correct results. The computer then learns how to do calculations to predict outcomes for new data it has never seen before.

The Problem: Credit Card Default

Many banks face a big problem: customers who take credit card loans but cannot pay them back. This is called "defaulting." For any bank, it is very important to predict if a customer will pay or not. Knowing the probability of a customer defaulting is actually more useful for the bank than just a "Yes" or "No" answer, as it helps them manage risks better.

Our Project Goal

In this project, we are using machine learning to solve this problem. We are building a model that looks at customer data to predict if they will pay their credit card bill next month or not.

Classification

Classification in machine learning can be "multiclass" (many categories) or "binary" (only two categories). In our case, we use **Binary Classification**. This means the output will only be one of two things:

1. **Y = (1)**: The customer will **not** pay next month.
2. **Y = (0)**: The customer **will** pay next month.

Data Analysis and Preprocessing

The dataset contains 30,000 clients and 23 features. However, real-world data often contains inconsistencies, missing values, and noise. Training a model on raw, "dirty" data leads to poor performance and low accuracy. To ensure high-quality predictions, we performed the following preprocessing steps:

1. Data Type Consistency

Machine learning models require numerical inputs for mathematical calculations. We ensured that all features were converted to **float64**. This consistency allows the model to process all values correctly without errors during the training phase.

2. Duplicate and Missing Value Handling

We checked the dataset for any duplicate entries to avoid redundant information, but no duplicates were found. For missing or undocumented values, specifically in the "Education" and "Marriage" features, we identified rows with values not defined in the dataset description. Since these represented only **1.3%** of the total data, we removed them without significantly affecting the dataset size.

```
df = df[df['X3'] != 6]
df = df[df['X3'] != 5]
df = df[df['X3'] != 0]

df['X3'].value_counts()   #removed all values (0,5,6)
```

```
X3
2.0    14030
1.0    10585
3.0     4917
4.0      123
Name: count, dtype: int64
```

```
df = df[df['X4'] != 0]

df['X4'].value_counts()     #removed all values (0)
```

```
X4
2.0    15806
1.0    13477
3.0      318
Name: count, dtype: int64
```

X3-> Education

X4-> Marital status

3. Outlier Management (Capping)

We identified outliers in features like credit limits and payment amounts. Instead of deleting these records—which would lose valuable information—we used a **Capping** strategy. Any value exceeding the statistical upper bound was set to that maximum limit. This prevents extreme values from distorting the model's learning process.

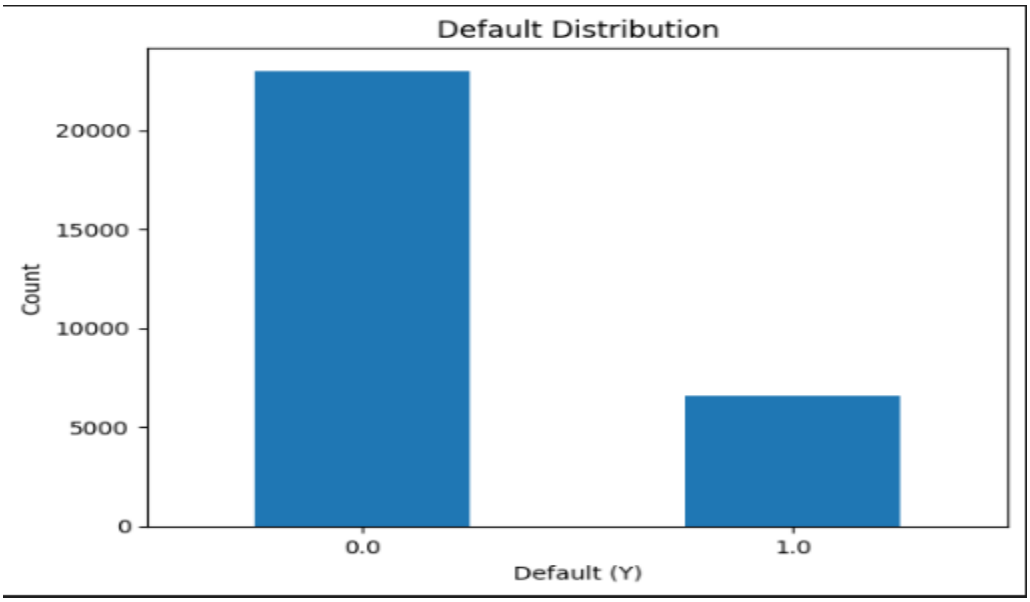
4. Handling Categorical Variables (One-Hot Encoding)

Features such as Gender, Education, and Marital Status are categorical. Assigning them simple numbers (like 1, 2, 3) can mislead the model into thinking there is a ranking or order (e.g., that "Female" is higher than "Male"). To solve this, we used **One-Hot Encoding**. This technique creates independent columns for each category, ensuring the model treats them as distinct labels without any bias or ranking.

Exploratory Data Analysis (EDA)

Before training the model, it is essential to analyze the relationships between features and the target variable, as well as the correlations among features themselves.

Default distribution:

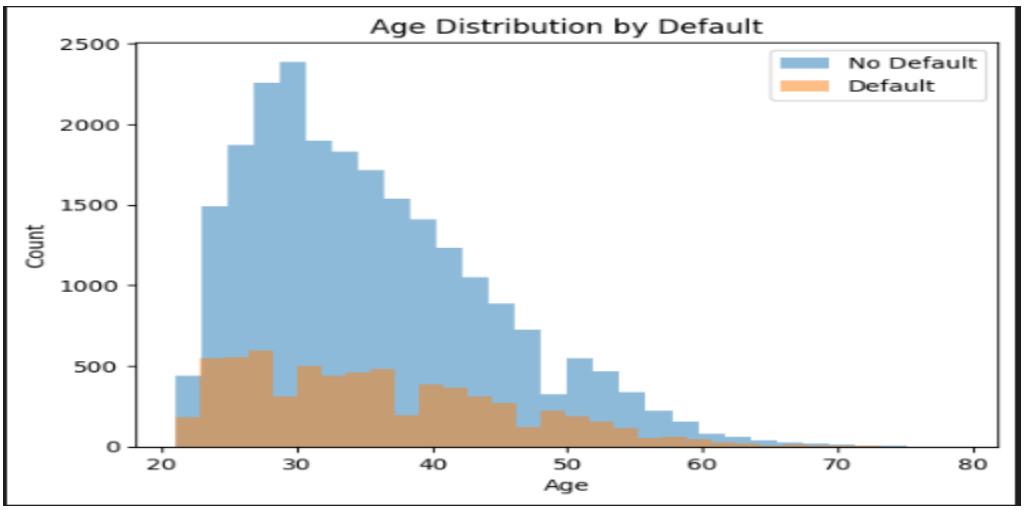


- The target variable is imbalanced, with approximately 77.7% of clients not defaulting and 22.3% defaulting.

- This imbalance may bias the model toward the majority class, making it less effective at identifying default cases.

- To fix this, we will use methods like SMOTE or set `class_weight='balanced'` in the model.

Age distribution by default status:

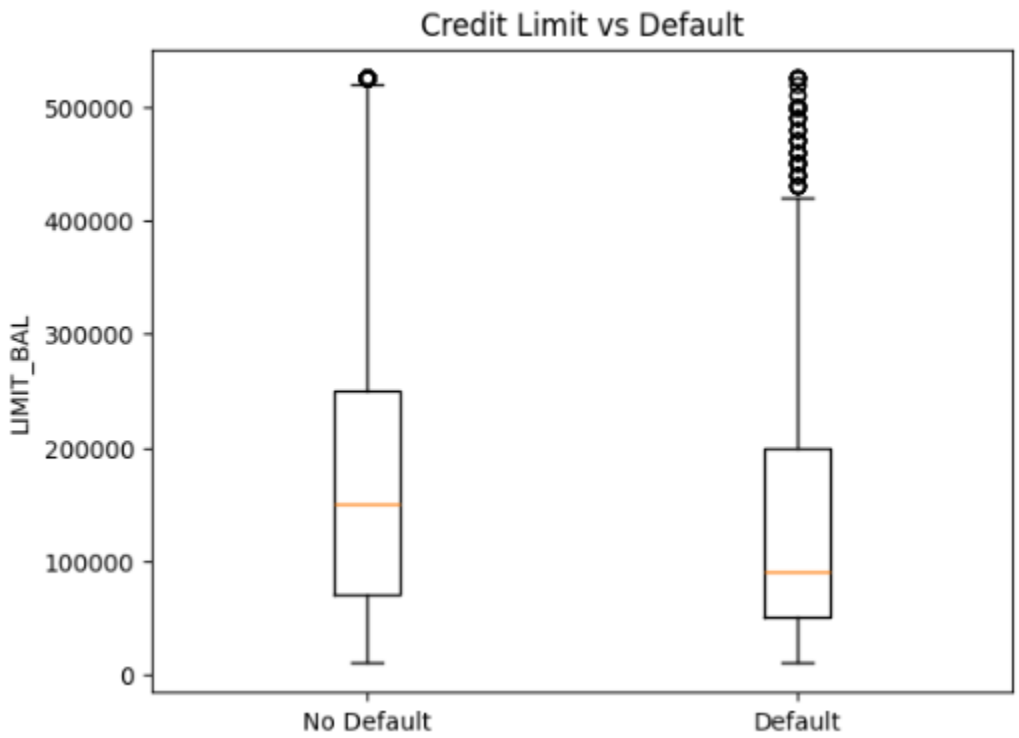


- Most credit card users are between 25 and 35 years old.

- Default cases appear across all age groups but are more common among younger clients because they make up a larger portion of the dataset.

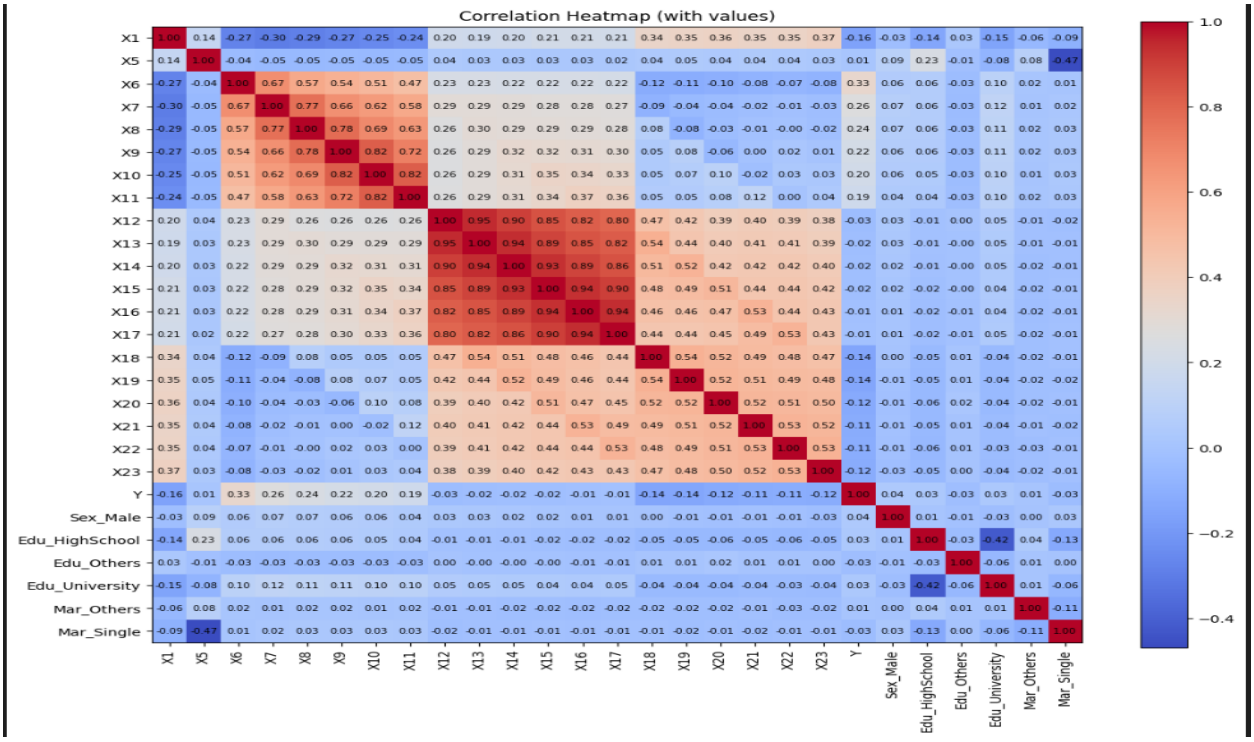
- As age increases (above 50), the number of both default and non-default clients decreases significantly.

Credit limit distribution by default status:



- Clients who did not default generally have higher credit limits compared to those who defaulted.
- This suggests that customers with greater financial trust from the bank are less likely to fail in repayment.
- Default cases are associated with lower credit limits and show higher variability, as indicated by the presence of outliers.

Correlation heatmap (before feature engineering):



• The heatmap shows relationships between features. Bill amounts and payment amounts are strongly related to each other.

• This may mean some variables are redundant because they describe similar financial behavior.

• Delay features (X6–X11), bill amounts (BILL_AMT1–BILL_AMT6), and payment amounts (PAY_AMT1–PAY_AMT6) are highly related. Because of this, we have created new features:

- `TOTAL_BILL` = sum of BILL_AMT1 to BILL_AMT6
- `TOTAL_PAY` = sum of PAY_AMT1 to PAY_AMT6

• Financial features have a stronger relationship with the target variable than demographic features. This means they are more useful for predicting default.

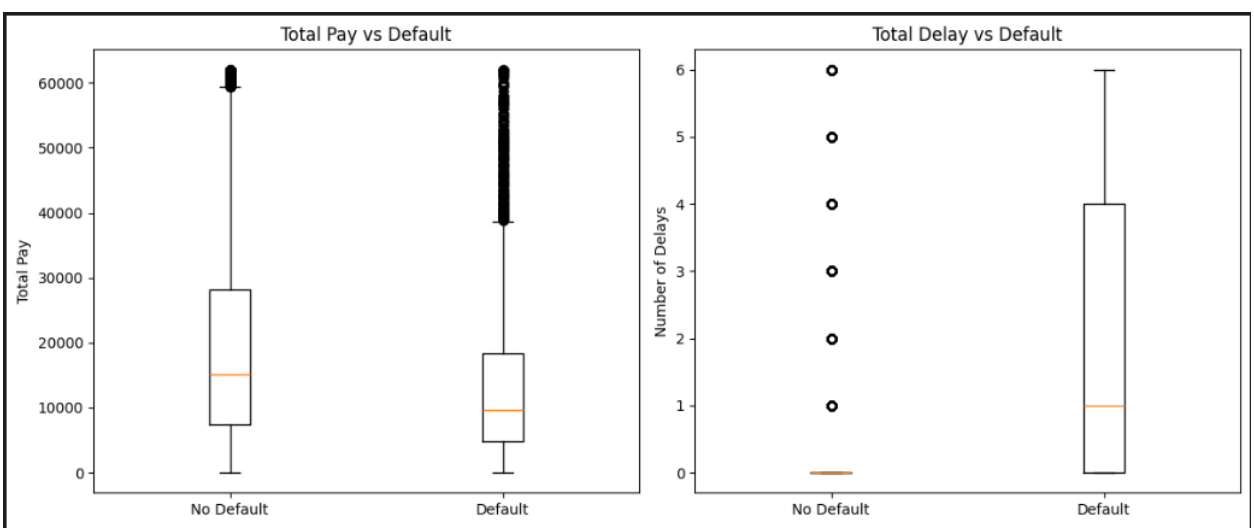
• We also created a new feature called `TOTAL_DELAY`, which counts how many months the client was late in payment. This helps summarize payment behavior in one value and makes it easier to understand.

Correlation heatmap (After feature engineering & feature selection):

• I selected features that have a good correlation with the target variable.

• Features with low correlation are not very useful for the model, so they were not included.

Total pay and Total delay vs Default



Total Delay vs Default:

• People who do not default usually have zero delays. This means they pay on time.

• People who default often have more delays, usually between 1 to 4 months.

• Even small delays (1–2 months) can be a strong sign that a person may default later.

Total Pay vs Default:

• People who do not default usually pay higher amounts.

• People who default tend to pay smaller amounts toward their bills.

• Some people pay large amounts but still default. This may happen because their total debt is even higher than what they pay.

Implementing First classifier:

For our first choice of classifiers we chose Logistic Regression which outputs probabilities for both classes using sigmoid function

```
#separating the target variable (Y) from the features (X)
y = df["Y"]
X = df[['X1', 'X6', 'X7', 'X8', 'X9', 'X10', 'X11',
        "TOTAL_BILL", "TOTAL_PAY", "TOTAL_DELAY"]]
```

```
from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import KFold , StratifiedKFold

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,stratify=y)
```

We used **train_test_split()** to split data for train and test , we used 80% for training and 20 for test% , **stratify = y** was used to preserve class distribution in the train and test dataset.

```
pipeline = Pipeline([
    ('model', LogisticRegression(random_state=42,class_weight='balanced'))
])
param_grid = {
    'model__max_iter': [1000, 10000, 100000],
    'model__C': [0.01, 0.1, 1, 10, 100],
    'model__penalty': ['l1', 'l2'],
    'model__solver': ['saga']
    }


cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
grid_search = GridSearchCV(estimator=pipeline, param_grid=param_grid, cv=cv, scoring='f1')
grid_search.fit(X_train, y_train)
```

Used LogisticRegression Class from sklearn to initialize the model with hyperparameter **class_weight = 'balanced'** to give different weights based on class distribution so this will help with the class imbalance.

Note (We tested both approaches (SMOTE and class_weight='balanced'). Both gave similar results, so we continued using the model with class weights)

used GridSearch to find the most optimal parameters for this model and dataset and used cross validation with 10 splits. This will divide data into 10 splits train on 9 and validate on the one left. This was the optimal parameters we obtained:

```
LogisticRegression
LogisticRegression(C=0.01, class_weight='balanced', max_iter=1000, penalty='l1',
                   random_state=42, solver='saga')
```

L1 penalty is a penalty on the absolute value of the coefficients, so weak features may have near zero or zero coefficients. C parameter is used to control regularization.

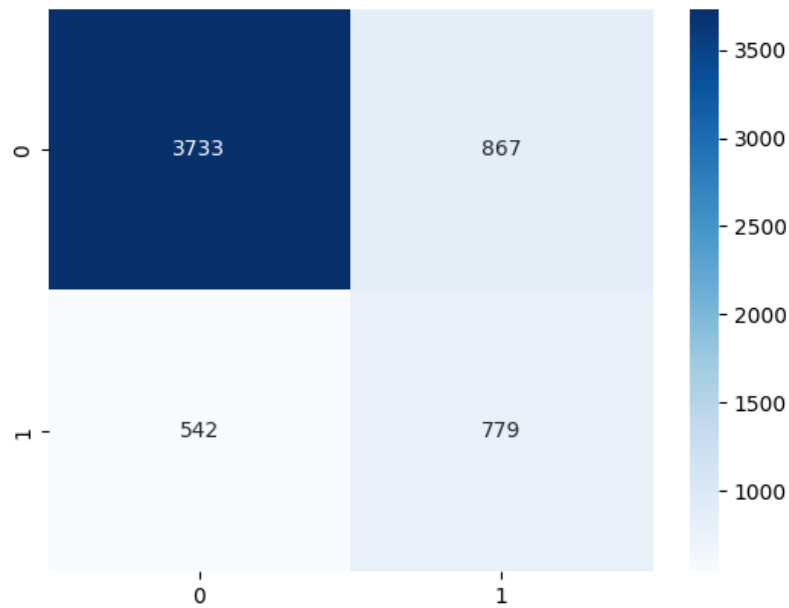
Here are the results of the model:

```
Classification Report:
              precision    recall  f1-score   support

         0.0       0.87      0.81      0.84      4600
         1.0       0.47      0.59      0.53      1321

    accuracy                           0.76      5921
   macro avg       0.67      0.70      0.68      5921
weighted avg       0.78      0.76      0.77      5921

ROC-AUC Score: 0.7530
F1-Score: 0.5251
Accuracy: 0.7620
Precision: 0.4733
Recall: 0.5897
```



As we can see the Logistic Regression model performs very badly on the minor class, and gives overall f1 score of 0.525 which is low. So, we conclude that logistic regression is not suitable for this dataset at all due to its low performance on the minority class.